Введение в язык Python

Лекция 1

Лектор: Половикова Ольга Николаевна

План

1. История языка Python.
2. Философия Python.
3. Интерпретатор. IDE.
4. Переменные, основные типы данных.
5. Модули и библиотеки.

Язык программирования Python

- *Python* – это интерпретируемый, объектно-ориентированный, кроссплатформенный, язык программирования высокого уровня, предназначенный для решения самого широкого круга задач
- *Python* – это универсальный, простой, гибкий и очень популярный язык программирования, который используется практически во всех областях современной разработки. С его помощью можно создавать веб-приложения, писать игры, выполнять математические и научные вычисления, заниматься анализом данных и др.



История языка Python

- Python был разработан **Гвидо ван Россумом** в конце 80-х - начале 90-х годов в Национальном исследовательском институте математики и компьютерных наук в Нидерландах.
- Python является производным от многих других языков, включая ABC, Modula-3, C, C ++, Algol-68, SmallTalk и другие языки написания сценариев.



Стоит отметить, что язык получил название «Python» не в честь вида змей, как ошибочно считают многие разработчики. Во времена разработки «Питона» Гвидо любил смотреть комедийное шоу «Воздушный цирк Монти Пайтона», поэтому и назвал своей проект в честь Монти Пайтона.

Python 3.8, 3.9, 3.10

Версия языка 2.0 - появилась в 2000г. и поддерживалась до 2020 года. Но, переход на версию 3 состоялся в 2008 году, и все новые модули и библиотеки пишутся для версии 3.

Последние версии

- Python 3.8 — 14 октября 2019 года
- Python 3.9 — 5 октября 2020 года
- Python 3.10 — 4 октября 2021 года

Область применения

Это скриптовый язык, который применяется для решения самого широкого спектра задач. Чаще всего Python применяют в работе с большими данными и разработке сайтов и мобильных игр. Он подходит и для создания десктопных и мобильных приложений.

- Web-разработка
- Графический интерфейс
- Базы данных
- Системное программирование
- Сложные вычислительные процессы
- Машинное обучение
- Автоматизация процессов
- Игровая индустрия

Python – интерпретируемый язык программирования

- *Компилятор* – это программа, которая переводит текст программы, написанный на языке программирования, в набор машинных кодов
- *Интерпретатор* – программа, реализующая обработку и выполнение исходного кода построчно по мере поступления кода на вход интерпретатора

В отличие от компилируемых языков, интерпретируемым для исполнения программы не нужен машинный код. Вместо этого программу построчно исполнят интерпретаторы.

Интерпретируемые языки не зависят от платформы, но они значительно уступают компилируемым языкам в скорости исполнения.

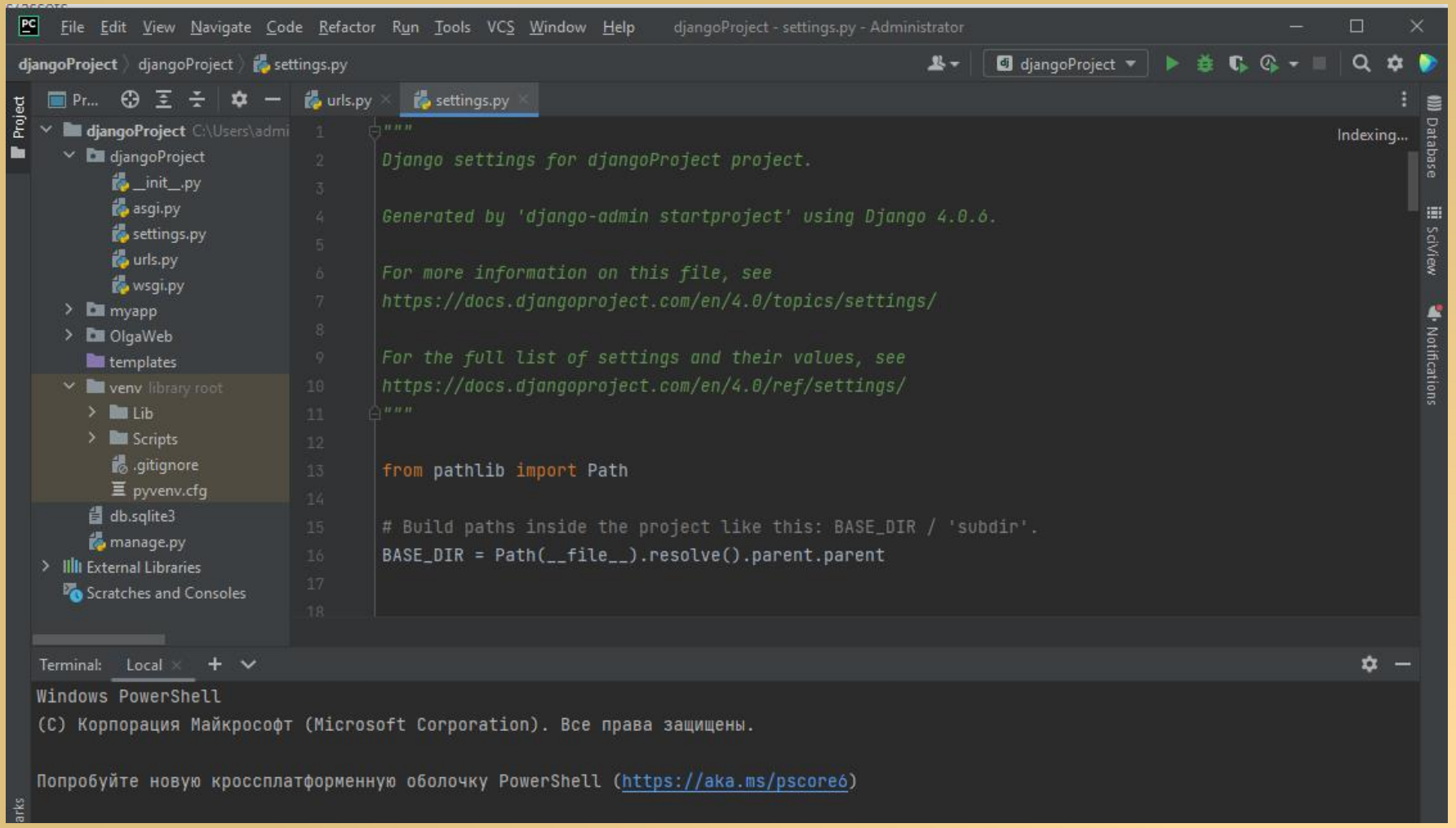


Особенности языка Python:

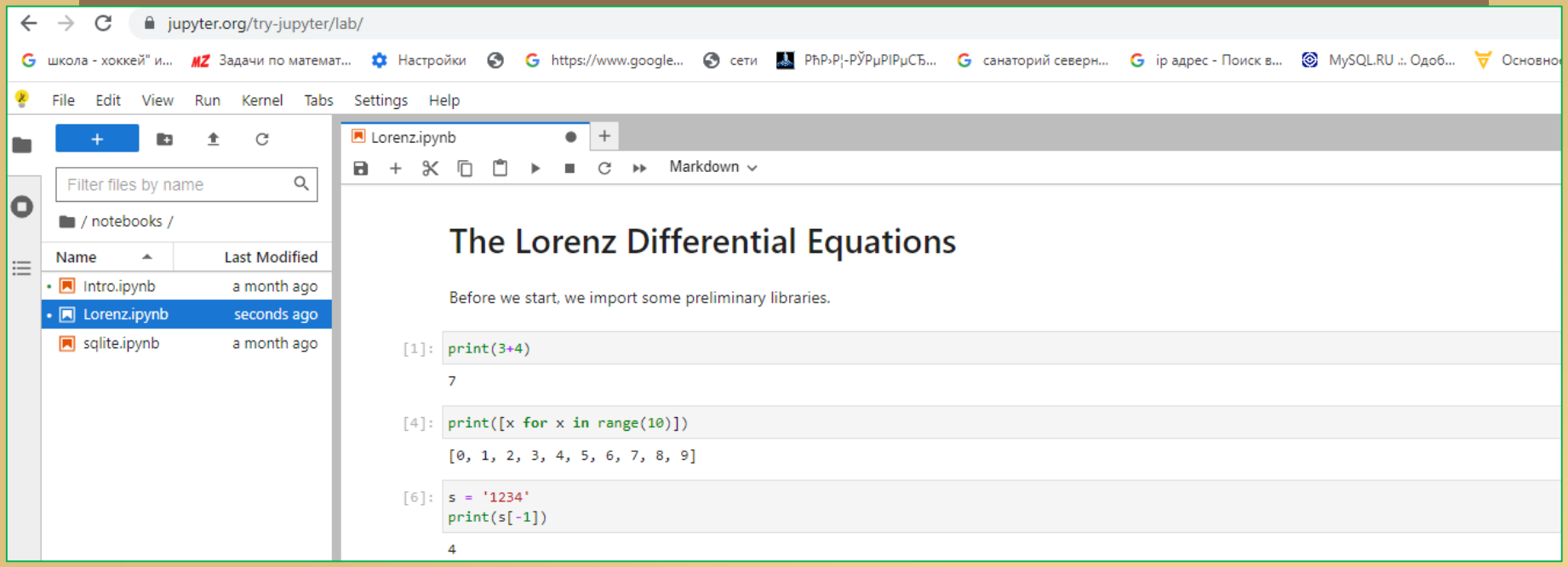
Основные архитектурные черты Python – динамическая типизация, автоматическое управление памятью, полная интроспекция, механизм обработки исключений, поддержка многопоточных вычислений, высокоуровневые структуры данных. Поддерживает разбиение программ на модули, которые, в свою очередь, могут объединяться в пакеты.

- Динамическая типизация – приём используемый в языках программирования, при котором переменная связывается с типом в момент присваивания значения, а не в момент объявления переменной.
- Управление памятью – это процесс эффективного распределения, выделения и координации памяти. Управление памятью также включает в себя очистку памяти от объектов, к которым больше нет доступа. Python управляет объектами с помощью подсчета ссылок. Это означает, что диспетчер памяти отслеживает количество ссылок на каждый объект в программе. Когда счетчик ссылок объекта падает до нуля, что означает, что объект больше не используется, сборщик мусора (часть диспетчера памяти) автоматически освобождает память от этого конкретного объекта.

IDE. Pycharm


File Edit View Navigate Code Refactor Run Tools VCS Window Help
djangoProject - settings.py - Administrator
djangoProject djangoProject settings.py
djangoProject
urls.py
settings.py
Indexing...
djangoProject C:\Users\admi
djangoProject
__init__.py
asgi.py
settings.py
urls.py
wsgi.py
myapp
OlgaWeb
templates
venv library root
Lib
Scripts
.gitignore
pyvenv.cfg
db.sqlite3
manage.py
External Libraries
Scratches and Consoles
"""
Django settings for djangoProject project.
Generated by 'django-admin startproject' using Django 4.0.6.
For more information on this file, see
https://docs.djangoproject.com/en/4.0/topics/settings/
For the full list of settings and their values, see
https://docs.djangoproject.com/en/4.0/ref/settings/
"""
from pathlib import Path
Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent
Terminal: Local
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.
Попробуйте новую кроссплатформенную оболочку PowerShell (https://aka.ms/pscore6)

Интерактивный блокнот. Jupyter Noteboook



Для того чтобы Вы лучше усвоили материал, я в презентации использовала примеры программ (программный код). Изображения с примерами программ были созданы как копии экрана (screenshot-ы) с работающего приложения Jupyter Notebook.

```
: #Здесь код программы
a = 10;
b = a ** 3
print('Вывод:', b + a, b, a)
```

← код программы

```
Вывод: 1010 1000 10
```

← результат работы (вывод)

Структура программы

Ступени иерархии в Python:

- программы делятся на модули
- модули содержат инструкции
- инструкции состоят из выражений
- выражения создают и обрабатывают объекты



Инструкция — минимальный самостоятельный элемент языка, определяющий действие, которое требуется выполнить.



Переменные и динамическая типизация

Переменная в Python это ссылка на объект!

- Каждая переменная должна иметь **уникальное имя**, состоящее из латинских букв, цифр и знаков подчеркивания
- Имя переменной не может начинаться с цифры.
- Следует избегать указания символа подчеркивания в начале имени
- **Регистр важен**!
- В качестве имени переменной **нельзя** использовать **ключевые слова**

Статическая типизация

```
int z;  // переменная z  целого типа
char w; // переменная w символьного типа
```

Динамическая типизация

```
a = 5
type(a)
```

```
int
```

```
b = 7.98
type(b)
```

```
float
```

Python предоставляет мощный набор объектных типов, встроенных в язык, что позволяет не тратить время на реализацию собственных структур данных. Если у вас нет потребности в специальных видах обработки, которые не обеспечиваются встроенными типами, то лучше использовать встроенные типы потому, что:

- **Встроенные типы объектов упрощают процесс разработки**
- **Встроенные типы обладают большей эффективностью и производительностью**

Тип определяет:
- область допустимых значений
- допустимые операции
- объём памяти
- формат хранения данных

Объекты программы:

В Python всё является объектами. Объекты делятся на две категории: **изменяемые** и **неизменяемые**.

Изменяемые объекты могут быть модифицированы, т.е. некоторые свойства объекта могут измениться \добавиться \удалиться, но это всё ещё будет тот же самый объект.

Неизменяемые, соответственно, изменить нельзя, совсем. Можно только создать изменённую версию, он это уже будет другой, отдельный объект.

- **int** (целый тип).
  В *Python* встроена арифметика для работы с целыми числами.
- **float** (вещественный тип).
  complex (комплексный тип).
- **bool** (логический тип).
  Для этого типа данных определены логические операции: конъюнкция and, дизъюнкция or, инверсия not.
- **str** (строковый тип).
- **tuple** (кортеж, неизменяемый список).

- Списки, list.
- Словари, dict.
- Множества, set.

Коллекции

Числа в Python

Числовой тип данных предназначен для хранения числовых значений. Это *неизменяемый тип данных*, что означает, что изменение значения числового типа данных приведет к созданию нового объекта в памяти (и удалению старого). Числовые объекты создаются, когда вы присваиваете им значение.

- Целые числа (**int**)
- Вещественные числа (**float**)
- Комплексные числа (**complex**)

```
a = 5
a
```

```
5
```

```
a = 8
a
```

```
8
```

```
k = 6
type(k)
```

```
int
```

```
n = 3.234
type(n)
```

```
float
```

```
s = 6+7j
type(s)
```

```
complex
```

Логический тип

Логический тип данных (**bool**):

- True (правда)
- False (ложь)

```
a = True
a
```

```
True
```

```
type(a)
```

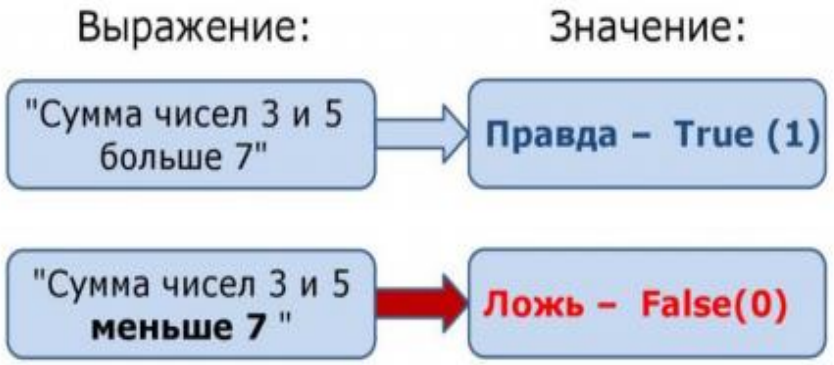
```
bool
```

```
6 == 7
```

```
False
```

```
6 > 4
```

```
True
```



Если результатом вычисления выражения может быть лишь истина или ложь, то такое выражение называется **логическим**.

Оператор	Значение	Выражение
==	Равно	A==B
!=	Не равно	A!=B
>	Больше	A>B
<	Меньше	A<B
>=	Больше или равно	A>=B
<=	Меньше или равно	A<=B

Строки

- ***Строки (str) – неизменяемые последовательности символов***
- Строки можно создавать используя кавычки или апострофы
- Как и любые последовательности строки поддерживают операции индексации и среза

```
s = 'Привет'
type(s)
```

```
str
```

```
st = "Привет"
type(s)
```

```
str
```

Чтобы обратиться к элементу последовательности, необходимо указать его индекс в квадратных скобках

Обратите внимание! Индексация начинается с нуля!

Обратная индексация (нумерация с конца)

```
s[0]
```

```
'П'
```

```
s[1]
```

```
'р'
```

```
s[-1]
```

```
'т'
```

Диапазоны

Диапазоны (range) – это диапазоны целых чисел с заданными начальным и конечным значением и шагом

Как и списки, кортежи и множества, диапазоны представляют собой последовательности и являются неизменяемыми

Для создания диапазона применяется функция `range()`:

```
range(<Начало>, <Конец>, <Шаг>)
```

- Первый параметр задает начальное значение – если он не указан, используется значение 0
- Во втором параметре указывается конечное значение. Следует заметить, что это значение не входит в возвращаемый диапазон
- Если параметр <Шаг> не указан, то используется значение 1

```
r = range(5)
r
```

```
range(0, 5)
```

```
for i in r:
    print(i, end = ' ')
```

```
0 1 2 3 4
```

```
for i in range(1, 10, 2):
    print(i, end = ' ')
```

```
1 3 5 7 9
```

- Важнейшим преимуществом диапазонов перед другими видами последовательностей является их компактность – вне зависимости от количества входящих в него элементов-чисел, диапазон всегда занимает один и тот же объем оперативной памяти.
- В диапазон могут входить лишь последовательно стоящие друг за другом целые числа – сформировать диапазон на основе произвольного набора чисел невозможно.

Стиль программирования на Python (Стандарт PEP-8)

- Он был составлен в 2001 году Гвидо ван Россумом, Барри Варшавой и Ником Когланом. Основная цель PEP 8 – улучшить читабельность и логичность кода на Python.
- PEP — это документ для сообщества, который описывает новые функции, предлагаемые для Python, и содержит такие аспекты языка, как дизайн и стиль
- Большинство IDE сред можно настроить для проверки / исправления кода под стандарт PEP8.
- Существует два класса инструментов, которые можно использовать для обеспечения соответствия PEP8: линтеры и автоформаттеры:

Линтеры

Это инструмент для проверки вашего кода на соответствие некоторым стилевым соглашениям в PEP8 (pycodestyle, flake8).

pycodestyle

Это инструмент для проверки вашего кода на соответствие некоторым стилевым соглашениям в PEP8.

Установите pycodestyle с помощью pip:

```
1. $ pip install pycodestyle
```

Вы можете запустить pycodestyle из терминала, используя следующую команду:

```
1. $ pycodestyle code.py
```

Результат:

```
1. code.py:1:17: E231 missing whitespace after ','
2. code.py:2:21: E231 missing whitespace after ','
```

Автоформаттеры

Автоформаттеры – это программы, которые автоматически реорганизуют ваш код для соответствия PEP 8 (autopep8, yapf, black).

```
$ pip install black
```

Его можно запустить из командной строки, как и в случае с линтерами. Допустим, вы начали со следующего кода, который не соответствует PEP 8, в файле с именем code.py:

```
for i in range(0,3):
    for j in range(0,3):
        if (i==2):
            print(i,j)
```

Затем вы можете запустить следующую команду через командную строку:

```
$ black code.py
reformatted code.py
All done! ✨ ? ✨
```

code.py будет автоматически приведён к следующему виду:

```
for i in range(0, 3):
    for j in range(0, 3):
        if i == 2:
            print(i, j)
```

Модули и библиотеки



Модули

Модуль – файл, содержащий определения и другие инструкции языка Python. Имя файла образуется путем добавления к имени модуля суффикса(расширения) '.py'.

Есть три типа модулей в Python:

1. Модули, написанные на Python (`.py`).
2. Модули, написанные на `C` и загружаемые динамически (`.dll` , `.pyd` , `.so` , `.sl` и так далее).
3. Модули, написанные на `C` , но связанные с интерпретатором.

```
import sys
print(sys.builtin_module_names)
```

```
('_ast', '_bisect', '_codecs', '_codecs_cn', '_codecs_hk', '_codecs_i
'_codecs_jp', '_codecs_kr', '_codecs_tw', '_collections', '_csv', '_d
'_functools', '_heapq', '_imp', '_io', '_json', '_locale', '_lsprof',
'_multibytecodec', '_opcode', '_operator', '_pickle', '_random', '_sh
'_sha256', '_sha512', '_sre', '_stat', '_string', '_struct', '_symtab
'_tracemalloc', '_warnings', '_weakref', '_winapi', 'array', 'atexit'
'binascii', 'builtins', 'cmath', 'errno', 'faulthandler', 'gc', 'iter
'math', 'mmap', 'msvcrt', 'nt', 'parser', 'signal', 'sys', 'time', 'w
'xxsubtype', 'zipimport', 'zlib').
```

Создание и использование своего модуля

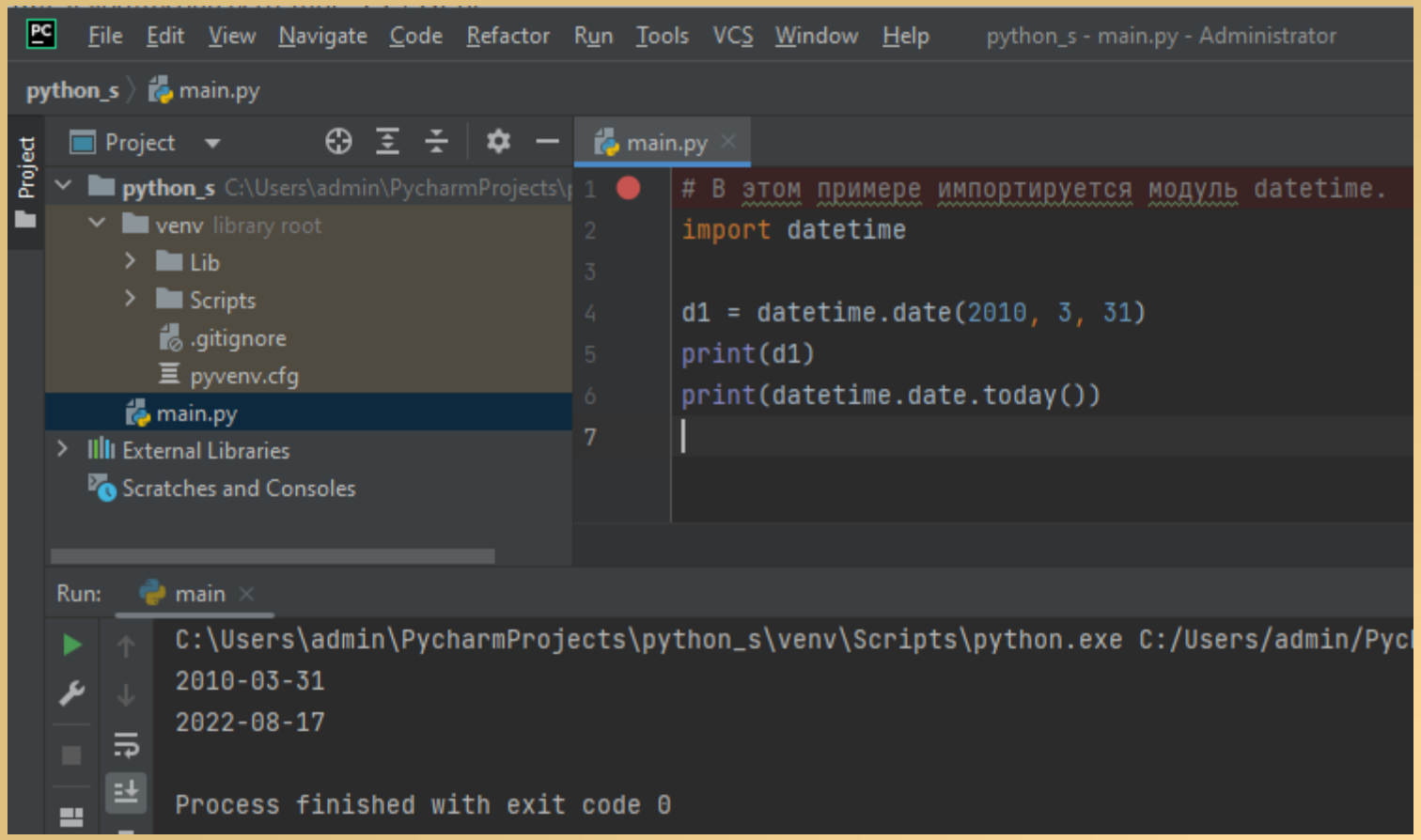
Модуль оформляется в виде отдельного файла с исходным кодом.

```
# создали файл fibo.py
def fib(n):
    a, b = 0, 1
    while b < n:
        print b,
        a, b = b, a+b
```

```
>>> import fibo
>>> fibo.fib(500) # 1 1 2 3 5 8 13 21 34 55 89 144
  233 377
```

Подключение и использование модуля

В программе на Python модуль представлен объектом-модулем, атрибутами которого являются имена, определенные в модуле:



В результате работы оператора import в текущем пространстве имен появляется объект с именем datetime

Подключение и использование модуля

Подключение модуля к программе на Python осуществляется с помощью оператора import. У него есть две формы: import и from-import:

import fibo

from fibo import fib, fib2

*from fibo import **

С помощью первой формы с текущей областью видимости связывается только имя, ссылающееся на объект модуля, а при использовании второй формы указанные имена (или все имена, если применена *) объектов модуля связываются с текущей областью видимости.

Если Вы собираетесь использовать функцию часто, то можно ее присвоить локальной переменной:

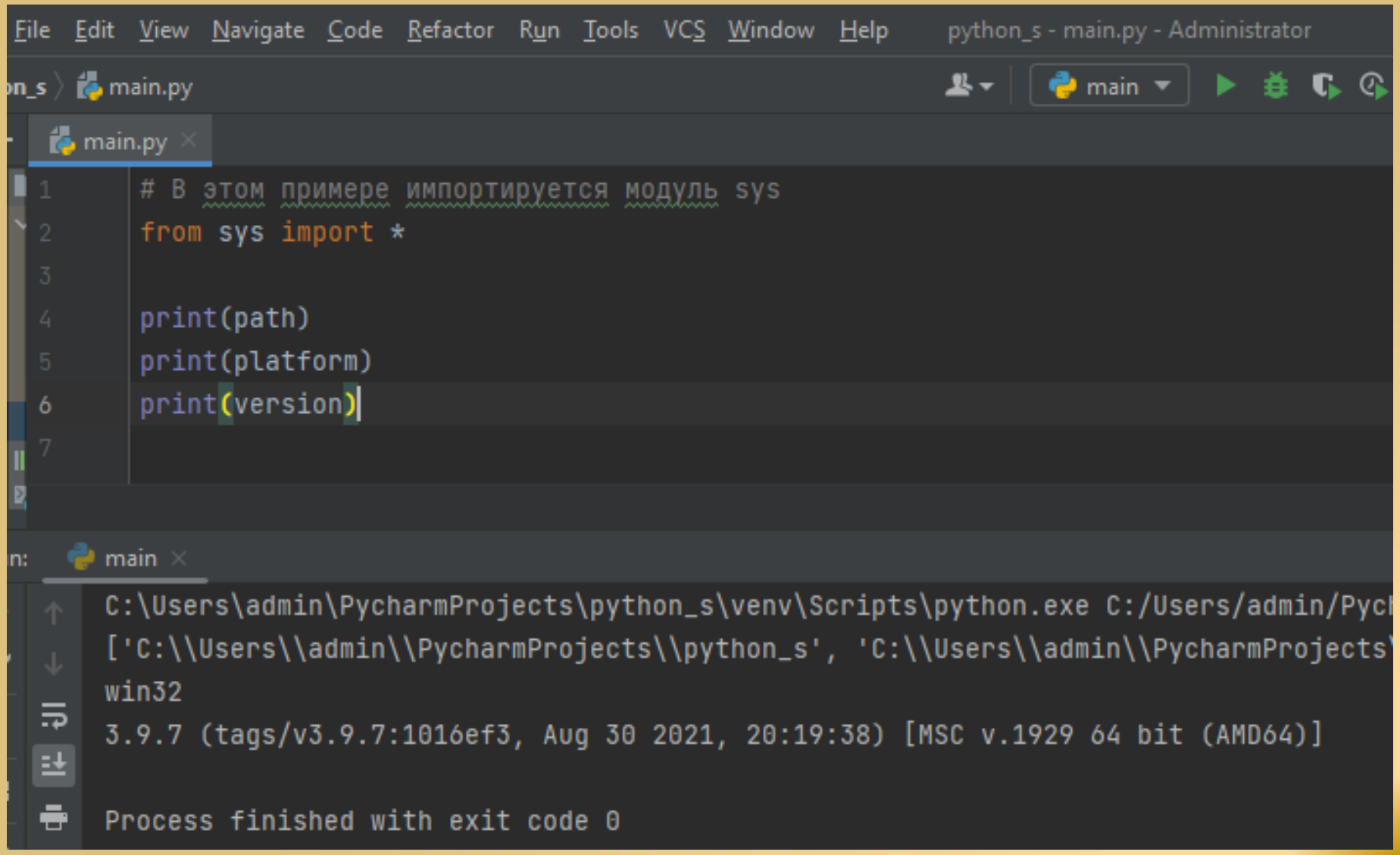
```
>>> fib = fibo.fib
>>> fib(500)
1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

Модуль sys

Модуль sys содержит информацию о среде выполнения программы, об интерпретаторе Python. Далее будут представлены наиболее популярные объекты из этого модуля:

exit([c])	Выход из программы. Можно передать числовой код завершения.
argv	Список аргументов командной строки.
sys.argv[0]	содержит имя запущенной программы, а остальные параметры передаются из командной строки.
platform	Платформа, на которой работает интерпретатор.
stdin, stdout, stderr	Стандартный ввод, вывод, вывод ошибок.
version	Версия интерпретатора.
setrecursionlimit(limit)	Установка уровня максимальной вложенности рекурсивных вызовов.
exc_info()	Информация об обрабатываемом исключении.

Модуль sys


File Edit View Navigate Code Refactor Run Tools VCS Window Help
python_s - main.py - Administrator
main.py
main
main.py
1 # В этом примере импортируется модуль sys
2 from sys import *
3
4 print(path)
5 print(platform)
6 print(version)
7
main
C:\Users\admin\PycharmProjects\python_s\venv\Scripts\python.exe C:/Users/admin/Pyc
['C:\\Users\\admin\\PycharmProjects\\python_s', 'C:\\Users\\admin\\PycharmProjects
win32
3.9.7 (tags/v3.9.7:1016ef3, Aug 30 2021, 20:19:38) [MSC v.1929 64 bit (AMD64)]
Process finished with exit code 0

Модуль OS

Программа на Python работает в операционной системе в виде отдельного процесса.
Функции модуля os дают доступ к различным значениям, относящимся к процессу и к среде, в которой он исполняется.
Одним из важных объектов, доступных из модуля **os**, является словарь переменных окружения environ.
В следующем примере можно получить переменную окружения PATH:

```
import os
PATH = os.environ['PATH']
```

Модуль tempfile

Программе иногда требуется создать временный файл, который после выполнения некоторых действий больше не нужен. Для этих целей можно использовать функцию TemporaryFile, которая возвращает файловый объект, готовый к записи и чтению.В следующем примере создается временный файл, куда записываются данные и затем читаются:

```
import tempfile
f = tempfile.TemporaryFile()
f.write("0"*100)          # записывается сто символов 0
f.seek(0)                 # уст. указатель на начало
                          #  файла
print len(f.read())       # читается до конца файла и
                          #  вычисляется длина
```

Как и следовало ожидать, в результате будет выведено 100. Временный файл будет удален, как только будут удалены все ссылки на его объект.

Модуль csv

Формат CSV (comma separated values - значения, разделенные запятыми) достаточно популярен для обмена данными между электронными таблицами и базами данных.

```
mydata = [(1, 2, 3), (1, 3, 4)]

import csv

f = file("my.csv", "w") # Запись в файл

writer = csv.writer(f)

for row in mydata:

    writer.writerow(row)

f.close()

reader = csv.reader(file("my.csv")) # Чтение из файла

for row in reader:

    print row
```

```
['1', '2', '3']

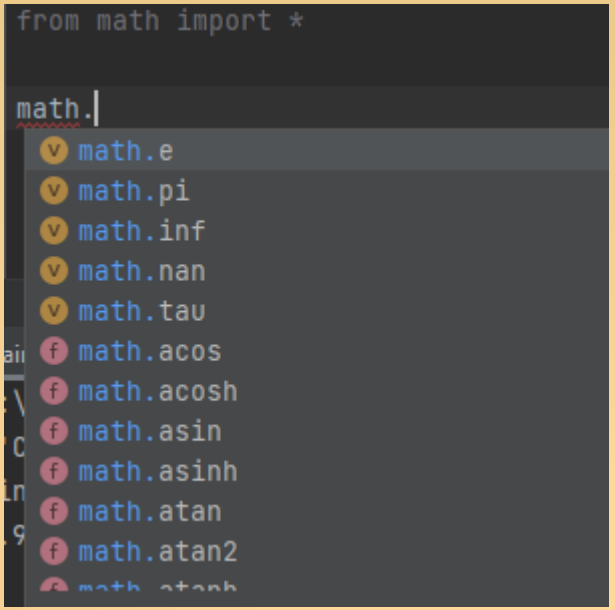
['1', '3', '4']
```

Математические модули

math: дает доступ к гиперболическим, тригонометрическим и логарифмическим функциям для действительных чисел,

cmath: позволяет работать с комплексными числами.

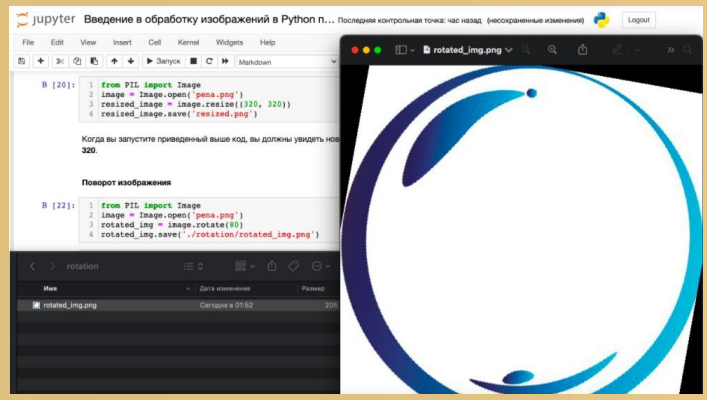
```
1  import math
2
3  math.ceil(1.001)     # returns 2
4  math.floor(1.001)    # returns 1
5  math.factorial(10)   # returns 3628800
6  math.gcd(10,125)     # returns 5
7
8  math.trunc(1.001)    # returns 1
9  math.trunc(1.999)    # returns 1
```



Библиотеки

NumPy – библиотека реализующая поддержку больших многомерных массивов и матриц, вместе с огромным набором высокоуровневых (и очень быстрых) математических функций для операций с этими массивами.

Flask – библиотека для создания быстрых веб-сервисов или *простых сайтов*. Flask – это микрофреймворк. Его задача – сделать ядро простым и расширяемым. Если необходима разработка *крупного веб-приложения*, то можно воспользоваться полноценным фреймворком, например, Django.



Pillow – это свободно распространяемая библиотека для работы с изображениями с открытым исходным кодом, которая добавляет вашему коду поддержку открытия, изменения и сохранения изображений в различных расширениях.



Библиотеки

Визуализация и анализ данных

- Matplotlib – это интерактивная библиотека для создания двумерных диаграмм. С ее помощью можно создавать качественные графики и диаграммы в нескольких форматах.
- Pandas – библиотека для высокопроизводительных и одновременно понятных структур данных и инструментов анализа данных.
- Scikit-Learn – библиотека предоставляет инструменты для анализа и добычи данных. Разнообразие алгоритмов, включая кластерный и факторный анализ, а также метод главных компонент. Может извлекать данные из изображений и текста.
- Seaborn – библиотека для создания статистических графиков. Построена на базе matplotlib и имеет интеграцию со структурами данных pandas.
- Keras – библиотека для машинного обучения, предоставляющая высокоуровневое API нейронной сети, работающее поверх TensorFlow, CNTK, Theano.
- SciPy – библиотека с разными модулями для оптимизации, линейной алгебры, интеграции и статистики.
- Pytorch – Содержит инструменты и библиотеки компьютерного зрения, натуральной обработки речи, глубокого обучения и др. Вычисления могут выполнятся на тензорах с помощью ускорения GPU. Можно создавать вычислительные диаграммы.
- TensorFlow – фреймворк для создания и работы вычислений, использующих тензоры. используется в нейронных сетях и глубоком обучении. TensorBoard – инструмент для визуализации моделей прямо в браузере.